Course Description

Zhuhai College of Science and Technology

College of Computer Science

Intelligent Science and Technology

ZhouWeijie

Date: January 9, 2026

Explanation of Credit Conversion and Academic Workload

1. Explanation of Credit Conversion between China and Germany

In Germany, the European Credit Transfer and Accumulation System (ECTS) is the standard system used in higher education to quantify academic workload. For international students, it is often necessary to convert domestic credits (CP) into ECTS credits to ensure compatibility and comparability across countries.

To standardize this conversion, many institutions refer to official methodologies. One widely accepted reference is the conversion approach used by Freie Universität Berlin (Conversion of Credit Points (CP) into ECTS). The formula for credit conversion is as follows:

$$\text{ECTS} = \text{CP} \times \frac{60 \times \text{Years of degree}}{\text{Total credits of degree}}$$

Where:

- **CP** refers to the domestic credit points of a course.
- **Total credits of degree** refers to the overall credits required for graduation.
- **Years of degree** denotes the official duration of the program in years.
- The resulting value is the equivalent ECTS credit.

The factor

$$\frac{60 \times \text{Years of degree}}{\text{Total credits of degree}}$$

is known as the **Credit Conversion Factor**, which can be pre-calculated to simplify further conversions.

Example Calculation

In the Intelligent Science and Technology program at Zhuhai College of Science and Technology, the official duration of the Bachelor's degree is **4 years**, and the minimum number of credits required for graduation is **165 CP**. Therefore, the credit conversion factor can be calculated as:

$$\textbf{Credit Conversion Factor} = \frac{60 \times 4}{165} \approx 1.45$$

Accordingly, in this document the ECTS value of each course is presented using the above factor, and the results are rounded to a practical level of precision where appropriate.

Remark on Academic Workload

It should be noted that the domestic credit system at Zhuhai College of Science and Technology primarily counts *contact hours*, including lectures, laboratories, and concentrated practical training. Independent academic workload such as self-study, homework, project preparation, and examination review is generally not explicitly quantified in the credit calculation.

In contrast, ECTS is designed to reflect the *total academic workload* of a student, including both contact hours and independent study. Under the ECTS framework, one ECTS credit typically corresponds to approximately 25–30 hours of total workload.

Therefore, while the calculated conversion factor for this Bachelor's program is approximately **1.45**, the presentation of ECTS values in this course description may be interpreted as being close to the commonly used approximation of **about 1.5 ECTS per domestic credit** when considering the unrecorded independent workload.

2. Relationship Between Credits and Class Hours

According to the general credit–class hour conversion rules of Zhuhai College of Science and Technology:

- **Theoretical courses**: 1 credit corresponds to 16 class hours.
- **Practical courses** (laboratory / computer practice, etc.): 1 credit corresponds to 32 class hours.
- **Concentrated practical modules** (internship / bachelor thesis, etc.): 1 credit corresponds to 1 week.

For completeness, the minimum graduation requirement for the Intelligent Science and Technology program is **165 credits**.

Contents

1 M100 Mathematics and Statistics **1**
1.1 Advanced Mathematics AI . . . 1
1.2 Advanced Mathematics AII . . . 2
1.3 Linear Algebra A . . . 2
1.4 Discrete Mathematics . . . 3
1.5 Probability Theory and Mathematical Statistics B . . . 3
1.6 Application of Stochastic Process . . . 4
1.7 Scientific Computing and MATLAB Language . . . 5
1.8 Artificial Intelligence Calculation Method I . . . 6

2 M200 Foundations of Computer Science **6**
2.1 Program Development Foundation . . . 7
2.2 Introduction to Computer . . . 8
2.3 Principles of Computer Composition . . . 8
2.4 Algorithms and Data Structures . . . 9
2.5 Operating Systems . . . 10
2.6 Introduction of Database . . . 11
2.7 Computer Network . . . 12
2.8 Data Engineering . . . 13
2.9 Compiling Principle . . . 14
2.10 Secrets of Algorithms and Programs . . . 15

3 M300 Fundamentals of Artificial Intelligence and Machine Learning **16**
3.1 Artificial Intelligence . . . 17
3.2 Introduction to Brain and Cognitive Science . . . 17
3.3 Artificial Intelligence . . . 18
3.4 Machine Learning . . . 19
3.5 Artificial Neural Network . . . 20
3.6 Computer Vision . . . 21
3.7 Intelligent Sensing and Detection Technology . . . 22
3.8 Natural Language Processing . . . 23
3.9 Pattern Recognition . . . 24
3.10 Artificial Intelligence and Information Society . . . 25
3.11 Intelligent Robot . . . 26

4 M400 Projects, Practice and Course Designs **27**
4.1 Course Project: Programming Development Foundation . . . 28
4.2 Course Design of Data Processing . . . 28
4.3 Course Project of Data Structures . . . 29
4.4 Machine Learning Curriculum Design . . . 30
4.5 Intelligent System Curriculum Design . . . 31
4.6 Comprehensive Course Design of Specialty . . . 32

5 M500 Programming Languages and Software Development **33**
5.1 UI Design . . . 34
5.2 Python Programming Design . . . 34
5.3 Object-Oriented Programming in Java . . . 36
5.4 Matlab language and its application . . . 36
5.5 Java Web Development Technology . . . 37
5.6 Mobile Application Development Technology . . . 38

1 M100 Mathematics and Statistics

Course Information Overview

Course Name	**Credits**	**ECTS**
Advanced Mathematics AI	5.0	7.5
Advanced Mathematics AII	5.0	7.5
Linear Algebra A	3.0	4.5
Discrete Mathematics	3.0	4.5
Probability Theory and Mathematical Statistics B	2.0	3.0
Application of Stochastic Process	2.0	3.0
Scientific Computing and MATLAB Language	2.0	3.0
Artificial Intelligence Calculation Method I	2.0	3.0
Total	**24.0**	**36.0**

1.1 Advanced Mathematics AI

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Advanced Mathematics AI	80	Compulsory	5.0	7.5
Semester: 2022-2023/1		**Teaching Method**: Lecture(100%)		

Course Contents:
• Functions and limits: number sets; elementary functions; epsilon–delta definition; classification of discontinuities; properties of continuous functions on closed intervals; uniform continuity.
• Derivatives and differentials: definition of derivatives and their geometric and physical interpretations; differentiation rules; higher-order derivatives; differentiation of implicit functions and parametric equations.
• Mean value theorems and applications: Rolle, Lagrange, Cauchy and Taylor theorems; monotonicity; extrema; concavity and convexity; inflection points; asymptotes; Taylor expansion and error estimation.
• Indefinite integrals: antiderivatives; basic integration formulas; substitution method; integration by parts; integration of composite and complex functions.
• Definite integrals: Riemann integral definition; integrals with variable upper limits; Newton–Leibniz formula.
• Applications of definite integrals: area and volume computation; modeling of physical quantities.
• Improper integrals: convergence and divergence criteria for infinite and improper integrals.
• Ordinary differential equations: separable equations; first-order linear equations; Bernoulli equations; Newton's iterative method.

Learning Objectives:
• Master the fundamental concepts and core properties of limits, continuity, derivatives and integrals of single-variable functions.
• Apply mean value theorems and Taylor expansions to function analysis and approximation problems.
• Proficiently use substitution and integration by parts to compute integrals and determine convergence of improper integrals.
• Solve basic ordinary differential equations and apply Newton's method to simple numerical optimization problems.
• Use definite integrals to model and solve practical problems in geometry and physics.

Assessment: Regular performance (30%); Mid-term exam (10%); Final exam (60%)

Course Materials: *Advanced Mathematics (8th Edition, Volume I)*, School of Mathematical Sciences, Tongji University, ISBN: 9787040515532, Higher Education Press

1.2 Advanced Mathematics AII

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Advanced Mathematics AII	80	Compulsory	5.0	7.5
Semester: 2022-2023/2		**Teaching Method**: Lecture(100%)		
Course Contents: • Differential calculus of multivariable functions: definitions; partial and higher-order partial derivatives; total differentials; gradients and directional derivatives; implicit function theorem; Jacobian matrix and coordinate transformations. • Multiple integrals: double and triple integrals; coordinate transformations; applications to volume, mass, centroid and moment of inertia. • Line and surface integrals: two types of line integrals and surface integrals; Green's theorem; Gauss's theorem; Stokes' theorem; divergence and curl of vector fields. • Infinite series: Fourier series expansion; convergence criteria; elementary applications in signal processing. • Advanced ordinary differential equations: second-order linear equations with constant coefficients; Euler equations; introductory stability analysis.				
Learning Objectives: • Extend single-variable calculus principles to multivariable functions and master partial derivatives, total differentials and gradients. • Apply coordinate transformations and integral theorems to solve vector field and multiple-integral problems. • Master Fourier series expansion and convergence analysis with preliminary signal analysis applications. • Solve higher-order ordinary differential equations and analyze stability in multivariable modeling contexts.				
Assessment: Regular performance (30%); Mid-term exam (10%); Final exam (60%)				
Course Materials: *Advanced Mathematics (8th Edition, Volume II)*, Department of Mathematics, Tongji University, ISBN: 9787040515549, Higher Education Press				

1.3 Linear Algebra A

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Linear Algebra A	48	Compulsory	3.0	4.5
Semester: 2022-2023/2		**Teaching Method**: Lecture(100%)		

Course Contents: • Determinants: definitions; expansion; algebraic properties; Cramer's rule. • Matrix operations: addition, multiplication, transpose, inverse; matrix rank and trace; introductory matrix decompositions. • Vector spaces: subspaces; linear dependence and independence; bases and dimensions. • Systems of linear equations: Gaussian elimination; homogeneous and non-homogeneous systems. • Eigenvalues and eigenvectors: characteristic polynomials; similarity; diagonalization of real symmetric matrices. • Quadratic forms: matrix representation; congruent transformations; canonical forms; positive definiteness. • Linear transformations and inner product spaces: kernels and images; orthogonality; Gram–Schmidt process; orthogonal projection; preliminary PCA applications.
Learning Objectives: • Master matrix operations and solution methods for linear systems. • Understand vector spaces, linear transformations and eigen-decomposition. • Perform quadratic form analysis and apply orthogonal projections. • Apply linear algebra techniques to dimensionality reduction and data modeling.
Assessment: Regular performance (30%); Mid-term exam (10%); Final exam (60%)
Course Materials: *Linear Algebra (March 2023 Edition)*, Department of Mathematics, Tongji University, ISBN: 9787040590254, Higher Education Press

1.4 Discrete Mathematics

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Discrete Mathematics	48	Compulsory	3.0	4.5
Semester: 2022-2023/2		**Teaching Method**: Lecture(100%)		

Course Contents: • Propositional and predicate logic: logical connectives; truth tables; equivalence transformations; normal forms; quantifiers and inference rules. • Set theory and relations: set operations; relations; equivalence relations and partitions; partial orders and Hasse diagrams. • Functions and mappings: injective, surjective and bijective functions; composite and inverse functions. • Graph theory: undirected and directed graphs; connectivity; paths and cycles; trees; minimum spanning trees (Prim and Kruskal algorithms); graph coloring. • Combinatorics: permutations and combinations; inclusion–exclusion principle; pigeonhole principle; recurrence relations; generating functions. • Boolean algebra: Boolean operations; Boolean expressions; applications in logic circuit design.
Learning Objectives: • Master formal reasoning in propositional and predicate logic. • Apply combinatorial and set-theoretic methods to counting and modeling problems. • Use graph theory to analyze network structures and algorithmic problems. • Apply Boolean algebra to logic circuits and discrete system modeling.
Assessment: Regular performance (40%); Mid-term exam (10%); Final exam (50%)
Course Materials: *Discrete Mathematics*, Hao Wu; Yanchun Liang; Xiongfei Li; Xijing Guo, ISBN: 9787302595638,

1.5 Probability Theory and Mathematical Statistics B

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Probability Theory and Mathematical Statistics B	32	Compulsory	2.0	3.0
Semester: 2023-2024/1		**Teaching Method**: Lecture(100%)		

Course Contents:
• Foundations of probability theory: sample spaces; events and operations; axiomatic definition of probability; classical and geometric probability; conditional probability; law of total probability and Bayes' theorem.
• Random variables: definitions of discrete and continuous random variables; distribution functions; common distributions and their numerical characteristics; distributions of functions of random variables.
• Multidimensional random variables: joint, marginal and conditional distributions; independence; distributions of sums, products and extrema.
• Numerical characteristics: mathematical expectation; variance; covariance; correlation coefficient; higher-order moments.
• Limit theorems: law of large numbers; central limit theorem and applications.
• Parameter estimation: method of moments; maximum likelihood estimation; unbiasedness, efficiency and consistency; interval estimation.
• Hypothesis testing: small probability principle; Type I and Type II errors; one-sample and two-sample tests; goodness-of-fit tests; contingency table independence tests.
• Regression and analysis of variance: linear regression models; parameter estimation and significance testing; one-way and two-way ANOVA.

Learning Objectives:
• Master axiomatic probability foundations and distribution laws of random variables.
• Analyze joint distributions of multivariate random variables and apply limit theorems in inference.
• Apply parameter estimation and hypothesis testing methods for statistical analysis.
• Use regression and variance analysis to study variable relationships and group differences.

Assessment: Regular performance (30%); Mid-term exam (10%); Final exam (60%)

Course Materials: *Probability Theory and Mathematical Statistics (5th Edition)*, Sheng Zhou; Shiqian Xie; Chengyi Pan, ISBN: 9787040554657,Higher Education Press

1.6 Application of Stochastic Process

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Application of Stochastic Process	32	Elective	2.0	3.0
Semester: 2025-2026/1		**Teaching Method**: Lecture(100%)		

Course Contents: • Fundamentals of stochastic processes: basic definitions; sample functions; finite-dimensional distributions; classification of stochastic processes (discrete-time and continuous-time, stationary and non-stationary); mean, covariance and correlation functions. • Markov chains: discrete-time and continuous-time Markov chains; Markov property; transition probability matrices; state classification; stationary distributions and limiting distributions. • Poisson processes: definition of counting processes; axiomatic characterization of Poisson processes; numerical characteristics; superposition and decomposition; compound Poisson processes and non-homogeneous Poisson processes. • Birth–death processes and queueing theory: transition rate matrices; balance equations; stationary distributions; M/M/1 queue model and performance analysis. • Brownian motion: definition; normality; independent increments; Markov property; numerical characteristics.
Learning Objectives: • Understand core concepts and numerical characteristics of stochastic processes. • Analyze state behavior and long-term properties of Markov chains and birth–death processes. • Apply Poisson and queueing models to dynamic system analysis. • Use stochastic process theory in communication, transportation and reliability modeling.
Assessment: Regular performance (40%); Final paper (60%)
Course Materials: *Introduction to Stochastic Processes (2nd Edition)*, Durrett (auth.),ISBN: 9780521765398 ,Cambridge University Press

1.7 Scientific Computing and MATLAB Language

Course Title	Hours	Course Type	Credit	ECTS
Scientific Computing and MATLAB Language	32	Elective	2.0	3.0
Semester: 2025-2026/1		**Teaching Method**: Lecture(100%)		

Course Contents: • Foundations of scientific computing: numerical methods; truncation and rounding errors; error propagation; significant digits; IEEE floating-point representation. • Numerical solution of linear systems: Gaussian elimination; LU decomposition; Jacobi and Gauss–Seidel iterations; conjugate gradient method. • Nonlinear equation solving: bisection method; Newton's method; secant method; Newton's method for nonlinear systems. • Numerical integration and differentiation: Newton–Cotes formulas; composite integration; Runge–Kutta methods; numerical differentiation. • Interpolation and fitting: Lagrange, Newton and spline interpolation; linear and nonlinear least squares fitting; error estimation. • Numerical methods for ODEs: Euler method; improved Euler method; Runge–Kutta methods; shooting method for boundary value problems. • MATLAB applications: matrix computation; data visualization; optimization and signal processing toolboxes; engineering simulation examples.
Learning Objectives: • Understand the core principles of numerical analysis, including error propagation, stability and convergence of numerical algorithms. • Apply interpolation, fitting and numerical integration techniques in modeling tasks. • Solve ordinary differential equations numerically and analyze stability and convergence. • Use MATLAB for numerical computation, visualization and engineering simulation.

Assessment: Regular performance (60%); Final exam (40%)
Course Materials: *Numerical Analysis (2nd Edition)*, Kendall Atkinson, ISBN: 9780471026414,John Wiley & Sons ; Online course materials as supplementary resources

1.8 Artificial Intelligence Calculation Method I

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Artificial Intelligence Calculation Method I	48	Elective	2.0	3.0
Semester: 2025-2026/1		**Teaching Method**: Lecture (33.3%) and Lab (66.7%)		

Course Contents:
• Mathematical modeling foundations for artificial intelligence, including probabilistic models, optimization-based formulations, and geometric representations.
• Vector and matrix operations: high-dimensional data representation; tensor operations; sparse data processing; matrix decompositions.
• Optimization methods: objective functions and constraints; convex sets and convex functions; gradient descent and variants (SGD, Adam); KKT conditions.
• Machine learning algorithms: linear regression; logistic regression; support vector machines; decision trees; Naive Bayes classifiers.
• Feature engineering: normalization and standardization; missing and outlier handling; feature selection and construction.
• Neural network fundamentals: activation functions; backpropagation; gradient vanishing and explosion mitigation; L1/L2 regularization; Dropout.
• Model evaluation: K-fold cross-validation; accuracy; precision; recall; F1-score; ROC curves and AUC.

Learning Objectives:
• Master the mathematical foundations of artificial intelligence models.
• Apply gradient-based optimization methods for model training.
• Understand machine learning algorithms and feature engineering techniques.
• Train and evaluate basic neural network models.
• Apply AI computational methods to classification, regression and approximation problems.

Assessment: Regular performance (40%); Final exam (60%)

Course Materials: *Introduction to Machine Learning and Practical Combat - Python Practical Application*, Leng Yuquan, Gao Qing, Yan Danqi, ISBN: 9787302600480, Tsinghua University Press.
Reference: *Pattern Recognition and Machine Learning (1st Edition)*, Christopher Bishop, ISBN: 9780387310732, Springer.

2 M200 Foundations of Computer Science

Course Information Overview

Course Name	**Credits**	**ECTS**
Program Development Foundation	4.0	6.0
Introduction to Computer	2.0	3.0
Principles of Computer Composition	3.5	5.25

Algorithms and Data Structures	3.5	5.25
Operating Systems	3.5	5.25
Introduction of Database	3.5	5.25
Computer Network	3.5	5.25
Data Engineering	2.5	3.75
Compiling Principle	3.0	4.5
Secrets of Algorithms and Programs	2.0	3.0
Total	**31.0**	**46.5**

2.1 Program Development Foundation

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Program Development Foundation	80	Compulsory	4.0	6.0
Semester: 2022-2023/1		**Teaching Method**: Lecture (60%) and Lab (40%)		

Course Contents:
• Introduction to the C++ programming language, including variables and data types, expressions, input and output operations, and control structures.
• Functions and modularization, covering function definitions and overloading, parameter passing mechanisms, recursion, and modular programming conventions.
• Arrays and pointers, including array operations, pointer definitions and applications, and the relationship between pointers and arrays.
• Object-oriented programming concepts, including classes and objects, encapsulation, constructors and destructors, and friend functions.
• Inheritance and polymorphism, including inheritance mechanisms, virtual functions, abstract classes, and operator overloading.
• Templates and the Standard Template Library (STL), including function and class templates, STL containers and iterators, and commonly used algorithms.
• Dynamic memory management and exception handling, including memory allocation and deallocation, exception capturing, and exception handling mechanisms.
• File operations, including text and binary file input/output and file stream applications.
• Programming paradigms overview, including imperative programming and basic functional programming concepts supported by C++ (e.g., lambda expressions).

Learning Objectives:
• Acquire proficiency in core C++ syntax and programming conventions, and effectively use control structures, functions, arrays, and pointers to solve fundamental programming problems.
• Understand object-oriented programming principles and apply classes, objects, inheritance, and polymorphism to design reusable and maintainable program modules.
• Use the STL standard library proficiently and develop foundational skills in template programming to improve development efficiency and code standardization.
• Master dynamic memory management and exception handling techniques and develop robust small- to medium-scale programs.
• Develop engineering-oriented programming competence, including version control usage and code documentation conventions.

Assessment: Regular performance (30%); Lab performance (30%); Final exam (40%)

Course Materials: *C++ Programming Tutorial (3rd Edition, General Version)*, Qian Neng, ISBN: 9787302224281, Tsinghua University Press; *Pro Git (2nd Edition)*, Scott Chacon, Ben Straub, ISBN: 9781484200773, Apress

2.2 Introduction to Computer

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Introduction to Computer	48	Compulsory	2.0	3.0
Semester: 2022-2023/2		**Teaching Method**: Lecture (33.3%) and Lab (66.7%)		

Course Contents:
• An overview of computer science, including its definition, major research areas, historical development, and frontier trends.
• Computer system organization, covering the von Neumann architecture, core hardware components, and hardware–software coordination mechanisms.
• Data representation principles, including numeral system conversion, character and numeric encoding, and information storage mechanisms.
• Computational thinking and algorithms, including core approaches to computational thinking, algorithm definitions, design principles, and basic algorithm implementation.
• Fundamentals of computer networks, including network classifications, the TCP/IP protocol stack, and core Internet applications.
• Introduction to databases, covering basic database concepts, data management fundamentals, and information system architectures.
• Computer security and information ethics, including security threats and protection strategies, ethical norms, and data protection principles.
• Frontier seminars focusing on interdisciplinary application topics and emerging technology trends.

Learning Objectives:
• Establish a comprehensive knowledge framework of computer science and understand the discipline's core areas and developmental logic.
• Master the composition and coordination mechanisms of computer systems, and understand data representation and encoding principles.
• Develop basic computational thinking skills and design simple algorithms to solve practical problems.
• Acquire foundational knowledge of computer networks, databases, and other core application domains and their application scenarios.
• Develop basic academic practice abilities, including literature retrieval and standardized citation, and cultivate awareness of information ethics and data protection.

Assessment: Regular performance (40%); Mid-term exam (10%); Final exam (50%)

Course Materials: *Introduction to Computers (2nd Edition)*, Zhang Kai, ISBN: 9787302560938, Tsinghua University Press

2.3 Principles of Computer Composition

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Principles of Computer Composition	64	Compulsory	3.5	5.25
Semester: 2023-2024/1		**Teaching Method**: Lecture (75%) and Lab (25%)		

Course Contents: • An overview of computer system organization, including hierarchical system structures, performance metrics, and architectural comparisons. • Digital logic fundamentals, including Boolean algebra, logic gates, and the design and analysis of combinational and sequential logic circuits. • Instruction set architecture concepts, including instruction formats, addressing modes, instruction types, and characteristics of CISC and RISC architectures. • CPU organization and design, including arithmetic logic units, control units, register files, instruction execution processes, and pipelining techniques. • Memory systems, including memory hierarchies, cache working principles, and the characteristics of main memory and secondary storage. • Input/output systems, including I/O interfaces, bus technologies, interrupts, Direct Memory Access (DMA), and peripheral control mechanisms. • Assembly language fundamentals, including instruction sets, addressing modes, and basic assembly program development. • Embedded system fundamentals, including microcontroller basics and introductory peripheral interfacing in simple embedded experiments. • Experimental practice, including circuit simulation, performance measurement, and microcontroller-based system development.
Learning Objectives: • Understand the hierarchical structure and operational principles of computer hardware systems, and analyze the functions and interactions of core components. • Master the design and analysis of digital logic circuits and apply Boolean algebra to solve basic logical problems. • Understand instruction set architectures and CPU execution mechanisms, including the principles and optimization strategies of pipelining. • Understand memory hierarchy structures and analyze the impact of cache mechanisms on system performance. • Acquire foundational knowledge of microcontroller basics and complete basic hardware experiments (e.g., simple interface and timing measurements) for understanding system-level performance.
Assessment: Regular performance (30%); Lab performance (20%); Final exam (50%)
Course Materials: *Principles of Computer Organization (3rd Edition)*, Tang Shuofei, ISBN: 9787040537561,Higher Education Press. **Reference**: *Embedded System Design and Practice (2nd Edition)*, Wang Tianmiao,ISBN: 9787302291047, Tsinghua University Press .

2.4 Algorithms and Data Structures

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Algorithms and data structures	64	Compulsory	3.5	5.25
Semester: 2023-2024/1		**Teaching Method**: Lecture (75%) and Lab (25%)		

Course Contents: • Fundamental concepts of data structures and algorithms, including Abstract Data Types (ADT) and algorithm complexity analysis. • Linear data structures, including sequential lists and linked list structures and operations. • Stack and queue data structures, including their definitions, operations, and typical applications. • String data structures, including string operations and pattern matching algorithms. • Tree and binary tree structures, including binary tree properties, traversal methods, binary search trees, and balanced binary trees. • Graph data structures, including graph representations, traversal algorithms, shortest path algorithms, and minimum spanning tree algorithms. • Searching and sorting algorithms, including implementation techniques and performance analysis. • Introductory algorithmic strategies, including greedy methods and dynamic programming ideas with typical applications (e.g., activity selection, knapsack-style problems) to support problem solving with data structures. • File organization and indexing methods, including B-tree and B+ tree indexing structures. • Algorithmic thinking extensions, including divide-and-conquer strategies and a high-level introduction to parallelizable problem decomposition. • Theoretical awareness, including a brief overview of computational feasibility (e.g., P vs. NP at a conceptual level) to understand practical limits of algorithm design.
Learning Objectives: • Master the definitions, principles, and implementations of common data structures, and select appropriate structures for specific problem scenarios. • Understand algorithmic complexity analysis and evaluate the performance trade-offs among different algorithms. • Implement classical searching and sorting algorithms and apply dynamic programming and greedy strategies to solve typical algorithmic problems. • Master core graph algorithms and advanced applications for solving real-world problems such as path planning and network optimization. • Develop awareness of algorithmic feasibility boundaries (e.g., an introductory view of P vs. NP) and understand how divide-and-conquer strategies can support scalable problem solving.
Assessment: Regular performance (30%); Lab performance (20%); Final exam (50%)
Course Materials: *Algorithms and Data Structures (C++ Language Version)*, Feng Guanghui,ISBN: 9787121359347, Electronic Industry Press . **Reference**: *Introduction to Algorithms (3rd Edition)*, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, The MIT Press, ISBN: 9780262033848.

2.5 Operating Systems

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Operating Systems	64	Compulsory	3.5	5.25
Semester: 2023-2024/2		**Teaching Method**: Lecture (75%) and Lab (25%)		

Course Contents: • Operating system fundamentals, including definitions and functions, classifications, architectural structures, and core characteristics. • Process and thread management, including process concepts, Process Control Blocks (PCB), state transitions, scheduling algorithms, and thread management. • Concurrent programming in operating systems, including multi-thread synchronization, atomic operations, and deadlock avoidance. • Synchronization and communication, including critical section problems, synchronization mechanisms, and inter-process communication methods. • Deadlock mechanisms, including deadlock conditions and prevention, avoidance, detection, and recovery strategies. • Memory management, including partition management, paging and segmentation, virtual memory, and page replacement algorithms. • Device management, including I/O device management, disk scheduling algorithms, and buffering techniques. • File management, including file classifications, directory structures, file system implementation, and file operations. • Security mechanisms, including multi-user permission models, security protection strategies, and process isolation mechanisms.
Learning Objectives: • Understand the core functions and operational mechanisms of operating systems, and analyze the implementation logic of process, memory, and file management modules. • Understand key mechanisms such as process scheduling, synchronization and communication, deadlock handling, and concurrent programming (multi-thread synchronization, atomic operations, deadlock avoidance), and analyze typical operating system problems. • Understand memory management and virtual memory principles, and evaluate the performance of page replacement algorithms. • Understand file system and device management implementations, and analyze optimization effects of disk scheduling algorithms. • Develop capabilities in analyzing system-level security mechanisms and process isolation strategies, and apply core operating system principles to practical system design.
Assessment: Regular performance (40%); Mid-term exam (10%); Final exam (50%)
Course Materials: *Computer Operating System Tutorial (4th Edition)*, Zhang Yaoxue,ISBN: 9787302501244 ,Tsinghua University Press. **Reference**: *Concurrent Programming: Principles and Practice (2nd Edition)*, Gregory R. Andrews,ISBN: 9780201498848, Addison-Wesley.

2.6 Introduction of Database

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Introduction of Database	64	Compulsory	3.5	5.25
Semester: 2023-2024/2		**Teaching Method**: Lecture (75%) and Lab (25%)		

Course Contents: • Overview of database systems, including database system components, DBMS functions, and data independence. • Data models, including Entity–Relationship (ER) models, relational models, and the three elements of data modeling. • SQL language, including data definition, manipulation, querying, views, and indexing. • Relational database design, including relational algebra, functional dependencies, normalization theory, and schema decomposition. • Security and integrity, including access control, encryption, and integrity constraints. • Concurrency control and recovery, including transaction ACID properties, locking mechanisms, and failure recovery techniques. • Database design methodology, including design procedures, ER diagram construction, and logical and physical design. • Advanced applications, including NoSQL databases, Web integration, and an introduction to big data systems. • Practical projects, including teamwork, database deployment, and application development.
Learning Objectives: • Understand the composition and operational principles of database systems and master core data modeling concepts and applications. • Use SQL proficiently for data definition, manipulation, and complex queries, and optimize SQL performance. • Master relational database design methods and apply normalization theory to design sound database schemas. • Understand database security, integrity, concurrency control, and recovery mechanisms to ensure data consistency and reliability. • Acquire foundational knowledge of NoSQL and big data systems, and develop skills in Web application integration and team-based project development.
Assessment: Regular performance (20%); Lab performance (30%); Final exam (50%)
Course Materials: *Principles and Applications of Databases (2nd Edition)*, Song Jinyu et al., ISBN: 9787302376125,Tsinghua University Press. **Reference**: *MongoDB: The Definitive Guide (3rd Edition)*, Kristina Chodorow, ISBN: 9781491954461,O'Reilly Media.

2.7 Computer Network

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Computer Network	64	Compulsory	3.5	5.25
Semester: 2024-2025/1		**Teaching Method**: Lecture (75%) and Lab (25%)		

Course Contents: • Fundamental concepts of computer networks, including definitions and classifications, performance metrics, and OSI/TCP-IP architectures. • Physical layer fundamentals, including transmission media, encoding and modulation techniques, and interface characteristics. • Data link layer mechanisms, including frame structures, MAC addressing, CSMA/CD, ARP, and error control. • Network layer concepts, including IP addressing, subnetting, routing protocols, and IP forwarding. • Transport layer protocols, including the characteristics of TCP and UDP, TCP connection management, and flow and congestion control. • Application layer protocols and programming, including HTTP, DNS, and FTP protocols, as well as socket programming. • Network security fundamentals, including encryption and authentication, firewall mechanisms, and attack prevention. • Practical applications, including protocol analysis tools, traffic measurement, and fault diagnosis. • Frontier technologies, including wireless and mobile networks, as well as multimedia communication.
Learning Objectives: • Understand layered network architectures and clearly explain the data transmission process across network layers. • Master core protocols of the TCP/IP stack and analyze their design principles and application scenarios. • Use socket programming proficiently to develop basic client–server network applications. • Understand foundational network security technologies and identify common network attacks and protection methods. • Develop skills in network configuration, protocol analysis, and fault troubleshooting, and evaluate network performance in wireless and mobile contexts.
Assessment: Regular performance (30%); Mid-term exam (10%); Final exam (60%)
Course Materials: *Computer Networks (7th Edition)*, Xie Xiren,ISBN: 9787121407610 ,Electronic Industry Press . **Reference**: *Computer Networking: A Top-Down Approach (8th Edition)*, James F. Kurose and Keith W. Ross, ISBN: 9780136681557, Pearson.

2.8 Data Engineering

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Data Engineering	48	Elective	2.5	3.75
Semester: 2024-2025/1		**Teaching Method**: Lecture (66.7%) and Lab (33.3%)		

Course Contents: • An overview of data engineering, including the definition of data engineering, the data lifecycle, and data classification and characteristics. • Data collection and preprocessing, including data acquisition methods, data cleaning, transformation, integration, and fusion. • Data storage technologies, including relational databases, distributed storage systems (e.g., Hadoop distributed architecture), virtualization technologies (Docker fundamentals), and the design of data warehouses and data lakes. • Data processing workflows, including ETL processes, batch and stream processing, and an introduction to big data frameworks. • Data visualization, including visualization principles, tool applications, and chart design and interpretation. • Data mining methods, including association rule mining, classification algorithms, and clustering algorithms. • Data governance, including data quality assessment, data security, and compliance. • Advanced applications, including statistical programming, interactive visualization, graph databases, and real-time stream processing. • Practical projects, including end-to-end data engineering workflows (Docker-based deployment) and team-based development.
Learning Objectives: • Understand the core concepts and workflows of the data engineering lifecycle, including data acquisition, preprocessing, storage, processing, and analytical applications, with an introduction to distributed storage and virtualization deployment. • Apply data preprocessing techniques to address common data quality issues such as missing values and outliers. • Understand the application scenarios of different data storage technologies, including distributed storage systems (e.g., Hadoop) and virtualization technologies (Docker fundamentals), and select appropriate storage solutions. • Develop foundational skills in data visualization and data mining, and use statistical programming tools to interpret and present data insights. • Understand the basic principles and application scenarios of real-time data processing and graph databases, and participate in Docker-based distributed data engineering projects.
Assessment: Regular performance (40%); Mid-term exam (10%); Final exam (50%)
Course Materials: *Principles and Applications of Big Data Technology (3rd Edition)*, Lin Ziyu, ISBN: 9787115556639, Posts and Telecommunications Press. **Reference**: *Docker in Action (3rd Edition)*, Jeff Nickoloff, ISBN: 9781617296361, Manning Publications; *Hadoop: The Definitive Guide (4th Edition)*, Tom White, ISBN: 9781491901687, O'Reilly Media.

2.9 Compiling Principle

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Compiling Principle	48	Elective	3.0	4.5
Semester: 2025-2026/1		**Teaching Method**: Lecture (100%)		

Course Contents: • Overview of compiler theory and practice, including compiler structure, compilation processes, and the division between front-end and back-end. • Formal languages and automata theory, including grammar definitions, the Chomsky hierarchy, finite automata, and regular languages. • Lexical analysis, including regular expressions, finite automaton implementations, and lexical analyzer construction. • Syntax analysis, including LL(1) grammars, LR parsing techniques, and recursive descent parsing. • Semantic analysis and intermediate code generation, including type checking, scope analysis, and intermediate representation construction. • Runtime system fundamentals, including memory management, stack frame structures, and parameter passing mechanisms. • Code optimization and target code generation, including local and loop optimizations, target code generation, and register allocation. • Advanced practice topics, including complete compiler implementation workflows, Static Single Assignment (SSA) form, and real-world compiler architectures. • Theoretical extensions, including foundations of computability and garbage collection algorithms.
Learning Objectives: • Understand the theoretical foundations of compiler design and master key concepts of formal languages and automata. • Understand the operational principles of each compilation phase and design lexical and syntax analyzers based on formal grammars. • Understand semantic analysis and intermediate code generation mechanisms, and analyze optimization techniques for improving target code efficiency. • Acquire the ability to design and analyze a simple programming language compiler framework, and understand the complete compilation workflow through experimental implementations and case studies. • Understand computability theory, garbage collection algorithms, and modern compiler architectures, providing theoretical support for programming language design and system optimization.
Assessment: Regular performance (50%); Final exam (50%)
Course Materials: *Principles and Practice of Compilation (3rd Edition)*, Huang Xianying, Tsinghua University Press, ISBN: 9787302532072. **Reference**: *Compiler Design: Principles and Practice (2nd Edition)*, Kenneth C. Louden, ISBN: 9780534950983, Cengage Learning; *The Garbage Collection Handbook: The Art of Automatic Memory Management (2nd Edition)*, Richard Jones et al., ISBN: 9781498702542, Chapman & Hall/CRC.

2.10 Secrets of Algorithms and Programs

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Secrets of Algorithms and Programs	32	Elective	2.0	3.0
Semester: 2025-2026/1		**Teaching Method**: Lecture (100%)		

Course Contents:
• Advanced theoretical foundations, including computability theory, complexity classes, and problem reduction techniques.
• Advanced algorithm design, including randomized algorithms, approximation algorithms, parameterized algorithms, and parallel algorithms.
• Complex data structures, including advanced tree structures and advanced applications of graph structures.
• Interdisciplinary algorithms, including data compression, cryptographic algorithms, and foundational machine learning algorithms.
• Algorithm optimization, including performance tuning, optimization for large-scale data processing, and engineering-oriented implementations.
• Innovative design topics, including analysis of competitive programming problems, algorithmic innovation strategies, and tracking frontier algorithms.
• Academic practice, including algorithm literature review, thematic report writing, and small-scale algorithm innovation projects.
• Frontier extensions, including selected modern topics such as scalable algorithmic paradigms and introductory neural network algorithms.

Learning Objectives:
• Understand the theoretical boundaries of algorithms and grasp core concepts of computability and complexity theory (e.g., complexity classes at an overview level and why some problems are computationally difficult).
• Master advanced algorithm design techniques and apply randomized and approximation algorithms to solve complex optimization problems.
• Understand application scenarios of interdisciplinary algorithms (especially foundational machine learning algorithms) and integrate algorithmic theory with engineering practice.
• Develop capabilities in algorithm optimization and innovative design, and design efficient algorithmic solutions for specific application contexts.
• Cultivate academic research and innovative thinking, track frontier developments in algorithms, and complete thematic research projects and reports based on algorithm literature review.

Assessment: Regular performance (60%); Final exam (40%)

Course Materials: *Algorithm Design and Analysis (4th Edition)*, Wang Xiaodong (Author), 2022, ISBN: 9787302599840, Tsinghua University Press.
Reference: *Machine Learning in Action*, Peter Harrington, ISBN: 9781617290183, Manning Publications.

3 M300 Fundamentals of Artificial Intelligence and Machine Learning

Course Information Overview

Course Name	**Credits**	**ECTS**
Artificial Intelligence	1.0	1.5
Introduction to Brain and Cognitive Science	2.0	3.0
Artificial Intelligence	2.5	3.75
Machine Learning	2.5	3.75
Artificial Neural Network	2.5	3.75
Computer Vision	2.5	3.75

Intelligent Sensing and Detection Technology	2.5	3.75
Natural Language Processing	2.5	3.75
Pattern Recognition	2.5	3.75
Artificial Intelligence and Information Society	2.0	3.0
Intelligent Robot	3.5	5.25
Total	**26.0**	**39.0**

3.1 Artificial Intelligence

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Artificial Intelligence	16	Elective	1.0	1.5
Semester: 2022-2023/1		**Teaching Method**: Lecture(100%)		

Course Contents:
• Fundamental Concepts: Definitions of AI; historical development; major schools (symbolism, connectionism, etc.); key milestones; and clarification of basic terminology (strong AI/weak AI).
• Typical Application Scenarios: Concrete application cases across domains, including medical AI (image recognition), financial algorithms (stock market operations), robotic perception, generative art ("Xiaobing" case), AI in education (intelligent tutoring systems), and health (remote diagnostic assistance).
• Human–Machine Relations and Ethics: Differences between human and machine intelligence; ethics of collective intelligence; data privacy and technological fairness; and potential impacts of AI on employment markets.
• Law and Compliance: Understanding core principles of GDPR (data minimization, purpose limitation); key points of China's Interim Measures for the Administration of Generative AI; and typical domestic cases of AI ethics governance (e.g., compliance rectification of algorithmic recommendation).
• Frontier Trends: Quantum computing + AI, AI singularity, and cultural adaptation of "machine Chinese people."
• Interaction and Practice: Case-based deliberation (ethical decision-making in autonomous driving), questionnaire survey analysis, thematic discussions, and reflective course writing.

Learning Objectives:
• Master core definitions, developmental trajectories, and typical application scenarios of AI, and understand differences between human and machine intelligence as well as ethical boundaries.
• Be able to distinguish core differences between AI and conventional programs, and analyze the advantages and limitations of AI applications across different domains.
• Be able to preliminarily interpret key points of AI-related laws and regulations, developing foundational compliance awareness.
• Be able to participate in critical discussions on AI's societal impacts (e.g., industrial transformation, the digital divide), forming a rational understanding of technology.
• Be able to complete thematic reports and course reflections, clearly presenting understanding of AI concepts, societal impacts, and compliance requirements.
• Be able to analyze the application value and potential risks of AI in education and health domains.
• Be able to understand practical requirements of AI ethics compliance through domestic cases.

Assessment: Regular performance (60%); Final exam (40%)

Course Materials: *Introduction to Artificial Intelligence (4th Edition)*, Lin Yaorui, Ma Shaoping (Authors), ISBN: 9787302519855, Tsinghua University Press, 2021

3.2 Introduction to Brain and Cognitive Science

Course Title	Hours	Course Type	Credit	ECTS
Introduction to Brain and Cognitive Science	32	Compulsory	2.0	3.0
Semester: 2022-2023/2		**Teaching Method**: Lecture(100%)		

Course Contents:
• Foundational Framework: The interdisciplinary framework of cognitive science (e.g., psychology, neuroscience), research paradigms, and the historical trajectory of the field.
• Neural and Perceptual Foundations: Human brain structure and functional specialization; neuronal mechanisms (synaptic transmission processes); multisensory integration (from low–mid–high-level perception); and fundamental principles of neural information processing.
• Core Cognitive Processes: Mechanisms of attention; memory systems (working/long-term memory); learning and cognitive development; problem solving and reasoning.
• Experiments and Modeling: Psychophysical experimental design; introduction to brain imaging techniques (ERP, fMRI); basic concepts of cognitive computational modeling; simple cognitive modeling cases (e.g., perceptron-based simulation of visual perception); fundamental methods for controlling variables in brain-science experimental design.
• Brain-Inspired Intelligence Connections: Mapping relationships between cognition and AI; cognitively inspired AI models; principles of brain-inspired computing.
• Human–Computer Interaction Adaptation: Cognitive load assessment; cognitive biases and interface design; principles of accessible design.
• Domestic Characteristic Applications: Characteristics of Chinese cognitive processing; brain–computer interface rehabilitation cases.
• Practice: Experimental design simulation (attention-/memory-related experiments); brain-inspired AI case analysis; course report writing.

Learning Objectives:
• Master the interdisciplinary theoretical framework and neural foundations of cognitive science, and become familiar with the physiological–psychological mechanisms underlying core cognitive processes.
• Be able to explain the internal logic of processes such as perception and memory, analyze experimental design and modeling approaches, and understand the interpretive logic of basic experimental data such as EEG signals.
• Be able to connect cognitive theories with AI models, providing theoretical support for brain-inspired intelligence and human–computer interaction design.
• Be able to apply cognitive principles to optimize intelligent systems, adapting to Chinese cognitive processing and domestic brain-inspired application scenarios.
• Be able to design basic cognitive experimental protocols, produce experimental reports and case analysis reports, and clearly present theoretical applications and analytical results.
• Be able to understand the basic workflow of neural information processing and explain the operating logic of simple cognitive models.
• Possess foundational competence in controlling variables when designing basic brain-science experiments.

Assessment: Regular performance (50%); Final exam (50%)

Course Materials: *Introduction to Brain and Cognitive Science (2nd Edition)*, Wang Zhiliang, ISBN: 9787563563448, Beijing University of Posts and Telecommunications Press, 2021

3.3 Artificial Intelligence

Course Title	Hours	Course Type	Credit	ECTS
Artificial Intelligence	48	Compulsory	2.5	3.75

Semester: 2023-2024/1	**Teaching Method**: Lecture (66.7%) and Lab (33.3%)
Course Contents: • Fundamental Theory: Agent modeling; categorization of environment types; problem formalization and search techniques (uninformed/heuristic/A* algorithms/adversarial search); basic computational-complexity analysis of AI algorithms (e.g., time-complexity comparisons of search algorithms). • Knowledge Representation and Reasoning: Propositional/predicate logic; basic Prolog syntax and simple logical inference; basic workflow for knowledge graph construction; Bayesian networks; and reasoning under uncertainty. • Planning and Decision-Making: Classical planning and simple cases (e.g., task decomposition in path planning); Markov decision processes (MDPs) and basic application scenarios (e.g., simple robot navigation); hands-on adversarial search (minimax + alpha–beta pruning). • Engineering Practice: Python implementation of classic AI algorithms; development of simple expert systems; AI experiment design and evaluation. • Ethics and Compliance: AI ethics theories; core requirements of China's Data Security Law; foundational awareness of international norms (GDPR). • Practice: Algorithm programming exercises, expert-system development, basic knowledge-graph applications, and experiment report writing.	
Learning Objectives: • Master the core theoretical framework and classic algorithms of AI, and become familiar with key methods in knowledge representation, reasoning, and planning. • Be able to formalize real-world problems as AI tasks and apply symbolic and data-driven methods to solve intelligent problems. • Be able to implement simple logical inference in Prolog, program foundational AI algorithms and prototype systems, and develop foundational engineering practice capability. • Be able to analyze AI ethical risks and compliance requirements and design compliant AI system solutions in alignment with domestic policies. • Be able to produce experiment reports and prototype system documentation, efficiently presenting algorithm implementations and application outcomes. • Be able to apply classical planning methods to solve simple task decomposition problems and establish basic MDP models. • Possess preliminary ability to analyze computational complexity of foundational AI algorithms.	
Assessment: Regular performance (60%); Final exam (40%)	
Course Materials: *Introduction to Artificial Intelligence (5th Edition)*, Wang Wanliang, ISBN: 9787040547169, Higher Education Press, 2020	

3.4 Machine Learning

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Machine Learning	48	Compulsory	2.5	3.75
Semester: 2023-2024/2		**Teaching Method**: Lecture (66.7%) and Lab (33.3%)		

Course Contents: • Fundamental Theory: Formalization of learning problems; core paradigms (supervised/unsupervised/semi-supervised/reinforcement learning); generalization ability and evaluation metrics (cross-validation); and dataset partitioning standards for machine learning projects (train/validation/test ratios). • Mathematical Foundations: Linear algebra (matrix operations), probability and statistics, and optimization theory (basic principles of gradient descent). • Core Algorithms: Supervised learning (regression/classification models); unsupervised learning (clustering/dimensionality reduction); ensemble learning (Bagging/Boosting); Gaussian mixture models (GMM) and the EM algorithm; and basic application cases of GMM in clustering (e.g., user behavior categorization). • Core Techniques: Feature engineering; foundational methods of feature selection (e.g., introductory filter-based and wrapper-based selection); bias–variance trade-off; regularization (L1/L2/Dropout); and basic methods for feature-importance analysis. • Specialized Applications: Time-series processing and imbalanced-data handling. • Engineering Practice: Data preprocessing; use of Scikit-learn/XGBoost toolkits; fundamentals of model deployment; and basic project development (including data preprocessing, model training, and evaluation). • Practice: Industrial/medical data modeling, algorithm comparison experiments, and course project development (including report writing).
Learning Objectives: • Master core machine learning paradigms, mathematical foundations, and classic algorithms (including GMM/EM), and become familiar with model evaluation and optimization methods. • Be able to understand the mathematical principles of core algorithms, translate real-world problems into machine learning tasks, and complete data preprocessing and feature engineering. • Be able to proficiently use mainstream toolkits to implement regression, classification, and clustering algorithms to address multi-scenario data problems. • Be able to evaluate model performance, mitigate overfitting/underfitting, and select algorithms appropriate to different data characteristics. • Be able to independently complete basic machine learning projects, produce project reports and model deployment plans, and clearly present data processing and model performance. • Be able to use GMM to solve simple clustering problems and master basic feature selection methods. • Possess the ability to partition datasets according to standards to ensure the validity of model evaluation.
Assessment: Regular performance (50%); Final exam (50%)
Course Materials: *Machine Learning (Python+Sklearn+TensorFlow2.0)*, Wang Hengjun, ISBN: 9787302561751, Tsinghua University Press, 2020

3.5 Artificial Neural Network

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Artificial Neural Network	48	Compulsory	2.5	3.75
Semester: 2023-2024/2		**Teaching Method**: Lecture (66.7%) and Lab (33.3%)		

Course Contents: • Fundamental Theory: Neuron models (MP/perceptron); activation functions (Sigmoid/ReLU); and principles of network-structure design. • Classic Networks: Multilayer perceptron (MLP), BP algorithm, autoencoders (AE), and simple applications of AE in data dimensionality reduction. • Deep Network Architectures: CNN, RNN, LSTM/GRU; understanding the basic structure of Transformers (multi-head attention concepts); and principles of deep generative models (VAE/GAN). • Training and Optimization: Loss functions; optimizers (comparisons among SGD/Adam/RMSProp); learning-rate scheduling; regularization (dropout/L1/L2); and comparative experiments on regularization methods (e.g., differences between Dropout and L2 regularization). • Engineering Practice: TensorFlow/PyTorch hands-on practice, data augmentation, and model compression/lightweighting. • Application Scenarios: Image classification/segmentation, text processing, and speech recognition. • Practice: Network construction and debugging; optimizer performance comparison experiments; visualization methods for analyzing overfitting in neural network models; domestic characteristic scenario applications (autonomous driving/medical imaging); and model training report writing.
Learning Objectives: • Master theoretical foundations of neural networks and the design logic of different architectures, and become familiar with training and optimization methods (including optimizer selection). • Be able to analyze adaptation scenarios for convolutional and recurrent networks, and independently build basic neural networks and deep generative models. • Be able to use deep learning frameworks to train, debug, and optimize models, adapting to multi-domain application scenarios. • Be able to apply frontier techniques such as transfer learning and few-shot learning to address domestic characteristic AI application problems. • Be able to produce model training reports and technical documentation, clearly presenting network design and comparative experimental results. • Be able to use autoencoders to complete simple data dimensionality reduction tasks and compare the effects of different regularization methods. • Possess preliminary ability to visualize and analyze overfitting phenomena in neural networks.
Assessment: Regular performance (30%); Lab performance (20%); Final exam (50%)
Course Materials: *Neural Networks and Deep Learning*, Qiu Xipeng, ISBN: 9787111698811, China Machine Press, 2022

3.6 Computer Vision

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Computer Vision	48	Compulsory	2.5	3.75
Semester: 2024-2025/1		**Teaching Method**: Lecture (66.7%) and Lab (33.3%)		

Course Contents: • Fundamental Theory: Imaging models (camera/pinhole imaging), image digitization, color spaces (RGB/HSV), and fundamentals of camera calibration. • Classical Vision Algorithms: Filtering (mean/Gaussian/median), edge detection (Sobel/Canny), feature extraction (basics of SIFT/ORB matching), image segmentation, and foundational segmentation methods (e.g., threshold segmentation and introductory region growing). • Advanced Vision Techniques: Object detection (YOLO basics) and motion analysis (basic target tracking). • Deep Learning Integration: CNN-based vision applications, fundamentals of ResNet architecture, and multimodal vision fusion. • Engineering Practice: OpenCV hands-on practice; C++/Python programming; basic video processing (frame extraction and simple target tracking); and basic operations for implementing simple image stitching in OpenCV. • Application Scenarios: Medical image processing, industrial inspection, and foundations of autonomous-driving vision. • Practice: Implementation of vision algorithms (classical + deep learning); development of foundational industrial inspection projects; evaluation metrics for visual feature matching (e.g., matching-accuracy computation); and experiment report writing.
Learning Objectives: • Master the mathematical foundations of images and visual data and classical algorithms, and understand the integration logic between deep learning and vision. • Be able to analyze modeling methods for vision problems and implement basic algorithms such as filtering, edge detection, and object detection (including classical feature matching). • Be able to use OpenCV and deep learning frameworks to address foundational engineering problems in scenarios such as medical imaging and industrial inspection. • Be able to optimize basic performance of vision systems, balancing real-time requirements and accuracy. • Be able to produce technical reports and project prototypes, clearly presenting vision-algorithm implementations and application outcomes. • Be able to complete image segmentation tasks using foundational methods and implement simple image stitching with OpenCV. • Possess preliminary ability to evaluate the effectiveness of visual feature matching.
Assessment: Regular performance (30%); Lab performance (20%); Final exam (50%)
Course Materials: *Computer Vision*, Duan Xianhua; Xu Dan; Chen Jianjun (Compilers), ISBN: 9787560667447, Xidian University Press, 2023

3.7 Intelligent Sensing and Detection Technology

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Intelligent Sensing and Detection Technology	48	Compulsory	2.5	3.75
Semester: 2024-2025/1		**Teaching Method**: Lecture (66.7%) and Lab (33.3%)		

Course Contents: • Fundamental Theory: Sensor principles (physical/chemical/biological sensing); performance metrics (sensitivity/SNR/range); calibration methods; and common sensor error-analysis methods (e.g., distinguishing systematic error vs random error). • Sensor Types: Resistive/inductive/capacitive sensors; ultrasonic/Hall/photoelectric/thermocouple sensors; physical/biological/image sensors; and multimodal sensors. • Data Processing: Acquisition system design; sensor interface principles; signal digitization; filtering and feature extraction (time-domain/frequency-domain); and basic principles of sensor signal-conditioning circuits (e.g., introduction to amplification and filtering circuits). • Detection Methods: Threshold-based detection, pattern-recognition-based detection, and anomaly detection algorithms. • Multi-Sensor Fusion: Understanding principles and application scenarios of Kalman filtering; Bayesian fusion cases; and basic workflows of multi-sensor data fusion (data alignment → feature extraction → decision fusion). • Engineering Practice: Basic integration of sensor data with AI models; measurement error analysis; and system anti-interference techniques. • Application Scenarios: Biosignal processing (EEG/ECG), industrial inspection, and environmental monitoring. • Practice: Sensor selection and debugging; data acquisition programming; hands-on industrial inspection cases; and experiment report writing.
Learning Objectives: • Master sensor operating principles, classifications (including specific types), and calibration methods, and become familiar with data acquisition and preprocessing workflows. • Be able to process sensor data, extract time-domain and frequency-domain features, and apply intelligent detection methods to solve foundational multi-scenario problems. • Be able to select sensor types suitable for scenarios, design signal-conditioning circuits, integrate sensors with AI models, and develop simple intelligent sensing systems. • Be able to understand principles of multi-sensor fusion algorithms, evaluate fusion performance, and optimize detection accuracy and stability. • Be able to produce experiment reports and system design documentation, clearly presenting sensor data processing, system construction, and detection results. • Be able to understand the basic roles of sensor signal-conditioning circuits and master the foundational workflow of multi-sensor fusion. • Possess preliminary ability to analyze common sensor error types.
Assessment: Regular performance (50%); Final exam (50%)
Course Materials: *Sensor and Intelligent Detection Technology*, Luo Zhizeng; Xi Xugang; Gao Yunyuan (Editors), ISBN: 9787560656212, Xidian University Press, 2020

3.8 Natural Language Processing

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Natural Language Processing	48	Elective	2.5	3.75
Semester: 2024-2025/1		**Teaching Method**: Lecture (66.7%) and Lab (33.3%)		

Course Contents: • Fundamental Theory: Definitions of NLP; historical development (rules → statistics → deep-learning-driven approaches); core tasks; and linguistic foundations (lexicon/syntax/semantics). • Text Preprocessing: Chinese word segmentation (jieba/HanLP), part-of-speech tagging, text cleaning, corpus construction, and basic corpus annotation norms (e.g., POS tag symbol conventions). • Core Techniques: Language models (n-gram); word embeddings (Word2Vec/GloVe training and visualization); text classification; sentiment analysis; topic modeling (LDA); foundational methods for named entity recognition (e.g., rule-based entity extraction); and extractive approaches to simple text summarization (e.g., keyword-weighted summarization). • Deep Learning for NLP: RNN/CNN fundamentals; concept of attention mechanisms; and basic application scenarios of pretrained models (BERT/ERNIE). • Ethics and Bias: Identification of algorithmic bias (bias in Chinese text) and privacy-preserving techniques for Chinese text. • Engineering Practice: Basic use of NLP toolkits and adaptation to Chinese scenarios. • Application Scenarios: Intelligent customer service, public-opinion analysis, and fundamentals of machine translation. • Practice: Text classification projects, sentiment analysis experiments, word-embedding model training, and Chinese NLP application development.
Learning Objectives: • Master computational modeling methods for natural language and the core technical system of NLP, and become familiar with the particularities of Chinese NLP. • Be able to apply segmentation and POS tagging to process Chinese/English text and speech data, train simple word-embedding models, and analyze semantic similarity. • Be able to use NLP toolkits to implement foundational language-processing systems and adapt to domestic characteristic application scenarios. • Be able to identify and mitigate ethical risks in NLP models (e.g., bias and privacy leakage), evaluate model performance, and propose optimization solutions. • Be able to produce project reports and system prototypes, clearly presenting text processing and application outcomes. • Be able to complete simple named entity recognition tasks using foundational methods and follow corpus annotation norms. • Possess preliminary ability to generate simple text summaries.
Assessment: Regular performance (30%); Lab performance (30%); Final exam (40%)
Course Materials: *Practical Python Natural Language Processing: Core Technologies and Algorithms*, Tu Ming; Liu Xiang; Liu Shuchun, ISBN: 9787111602802, China Machine Press, 2018

3.9 Pattern Recognition

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Pattern Recognition	48	Elective	2.5	3.75
Semester: 2024-2025/2		**Teaching Method**: Lecture (66.7%) and Lab (33.3%)		

Course Contents:
• Fundamental Theory: Definitions of pattern recognition; system workflow (acquisition → preprocessing → features → classification/clustering → evaluation); and core tasks.
• Data Preprocessing: Standardization, normalization, and handling missing values/outliers.
• Feature Engineering: Feature extraction (statistical/transform/structural features), feature selection, dimensionality reduction (PCA/LDA hands-on practice), and foundational feature-extraction methods (e.g., introductory statistical and structural feature extraction).
• Core Algorithms: Classification (core ideas and applications of Bayesian decision theory, Bayesian classifiers, SVM, decision trees); application cases of Bayesian classifiers in simple text classification; clustering (K-Means, DBSCAN); EM algorithm; and ensemble learning (Bagging/AdaBoost).
• Evaluation Methods: Confusion matrix; accuracy/recall/F1; ROC/AUC; clustering validity evaluation (silhouette coefficient/DB index); and foundational error analysis for pattern recognition systems (e.g., distinguishing types of classification errors).
• Application Scenarios: Pattern recognition in images/biosignals (ECG)/text, and fundamentals of industrial defect recognition.
• Practice: Pattern recognition system design, industrial defect detection case analysis, and experiment report writing.

Learning Objectives:
• Master the theoretical foundations and system design workflow of pattern recognition, and become familiar with feature engineering and core algorithms.
• Be able to design complete pattern recognition systems, completing data preprocessing, feature engineering (e.g., PCA dimensionality reduction), and algorithm selection/implementation.
• Be able to apply pattern recognition methods to solve foundational real-data problems in images and biosignals.
• Be able to use clustering validity indices to evaluate models, optimize feature engineering and algorithm selection, and avoid overfitting.
• Be able to produce experiment reports and system design documentation, clearly presenting pattern recognition workflows, algorithm comparisons, and recognition results.
• Be able to use Bayesian classifiers to solve simple text classification problems and master foundational feature-extraction methods.
• Possess preliminary ability to analyze error types in pattern recognition systems.

Assessment: Regular performance (30%); Lab performance (30%); Final project (40%)

Course Materials: *Pattern Recognition*, Liu Mingtang, ISBN: 9787121406248, Electronic Industry Press, 2021

3.10 Artificial Intelligence and Information Society

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Artificial Intelligence and Information Society	32	Elective	2.0	3.0
Semester: 2024-2025/2		**Teaching Method**: Lecture(100%)		

Course Contents: • Fundamental Theory: Information society theory; social penetration of AI (work/education/health domains); and integration with the digital economy. • Ethical Theory: Core ethics (utilitarianism/deontological ethics); responsible innovation; and ethical dilemmas (algorithmic fairness/accountability). • Law and Compliance: Core requirements of China's Data Security Law and Personal Information Protection Law; understanding core ideas of the EU AI regulatory framework; and specific domestic AI governance policies (e.g., interpretation of key clauses in the Interim Measures for the Administration of Generative AI Services). • Core Issues: Algorithmic bias, data privacy, basic concepts of explainable AI (XAI), the digital divide, and AI's impacts on the labor market. • Domestic Characteristics: China's AI governance framework, the Digital China strategy, domestic ethics cases (e.g., compliance of algorithmic recommendation), and AI's foundational role in driving the digital economy (e.g., AI applications in the circulation of data factors). • Practice: AI ethics case analysis; thematic reports (literature review + debate); basic approaches to implementing algorithmic transparency (e.g., visualization of decision logic in simple algorithms); and written paper composition.
Learning Objectives: • Master pathways of AI social penetration and ethical/legal frameworks, and become familiar with domestic compliance requirements. • Be able to analyze AI ethical risks and societal impacts (e.g., the digital divide and labor market transformation) and propose ethics-and-compliance solutions for specific scenarios. • Be able to evaluate social consequences of AI systems, develop critical reflective capacity, and design responsible AI application solutions. • Be able to clearly articulate analyses and response strategies regarding AI's societal impacts through thematic reports, debates, and written papers. • Be able to interpret domestic AI governance policies and provide support for compliant deployment of AI applications. • Be able to interpret key clauses of domestic AI-related policies and analyze AI's foundational role in promoting the digital economy. • Possess preliminary ability to understand basic approaches to implementing transparency for simple algorithms.
Assessment: Regular performance (60%); Final exam (40%)
Course Materials: *Artificial Intelligence and Society (2nd Edition)*, Li Yanhong et al. (Authors), ISBN: 9787115598327, Posts and Telecommunications Press, 2022

3.11 Intelligent Robot

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Intelligent Robot	64	Compulsory	3.5	5.25
Semester: 2024-2025/2		**Teaching Method**: Lecture (75%) and Lab (25%)		

Course Contents:
• Fundamental Theory: Robot system architecture (mechanics/perception/control/decision-making) and classifications (industrial/medical/service robots).
• Core Technologies: Basic kinematic models (basic computations for forward/inverse kinematics), fundamentals of dynamics, path planning (A*/RRT algorithms), motion planning, and foundational algorithms for robot motion control (e.g., introductory PID parameter tuning).
• Control Methods: PID control, fundamentals of model predictive control, and visual servoing.
• AI Integration: Fundamentals of reinforcement-learning-based motion control; machine-learning-based object recognition; and multi-sensor data fusion (vision + force sensing + inertial sensing).
• Human–Robot Interaction: Intention recognition, safe interaction, and fundamentals of voice/tactile interaction.
• Engineering Practice: Fundamentals of the ROS framework (node creation/topic communication); principles of SLAM and basic mapping workflows; foundational principles of SLAM (e.g., collaborative logic of mapping and localization); team-based project development; and robot system debugging (simulation environments).
• Application Scenarios: Industrial welding/assembly, medical surgery/rehabilitation, and logistics robots.
• Practice: Robot programming exercises (ROS basics), path planning experiments, SLAM mapping fundamentals, preprocessing methods for robot sensor data (e.g., noise filtering and data calibration), and medical/industrial robotics case development.

Learning Objectives:
• Master intelligent-robot system architectures and core technical system, and become familiar with kinematics/dynamics and control methods (including mathematical modeling foundations).
• Be able to establish robot kinematic models and solve basic forward/inverse kinematics problems, completing mathematical modeling for path planning and motion planning.
• Be able to integrate AI methods into robotic systems; develop foundational ROS-based robot prototypes; and develop awareness of multi-sensor fusion and SLAM applications.
• Be able to design simple robot task plans, evaluate robot system performance, and optimize motion precision and interaction safety.
• Be able to produce technical reports and project prototypes, clearly presenting robot system design and simulation/hands-on outcomes.
• Be able to understand basic principles of PID control and master foundational methods for preprocessing robot sensor data.
• Possess preliminary ability to understand core SLAM logic and describe the collaborative process of mapping and localization.

Assessment: Regular performance (20%); Lab performance (30%); Final exam (50%)

Course Materials: *Introduction to Intelligent Robots*, Zhu Ming, ISBN: 9787302629053, Tsinghua University Press, 2023

4 M400 Projects, Practice and Course Designs

Course Information Overview

Course Name	**Credits**	**ECTS**
Course Project: Programming Development Foundation	2.0	3.0
Course Design of Data Processing	2.0	3.0
Course Project of Data Structures	2.0	3.0
Machine Learning Curriculum Design	2.0	3.0

Intelligent System Curriculum Design	2.0	3.0
Comprehensive Course Design of Specialty	2.0	3.0
Total	**12.0**	**18.0**

4.1 Course Project: Programming Development Foundation

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Course Project: Programming Development Foundation	32	Compulsory	2.0	3.0
Semester: 2022-2023/2		**Teaching Method**: Lecture(100%)		

Course Contents:
• Fundamental Syntax and Development Environment Application: Become familiar with C++ development environment configuration; master fundamental syntax such as variables, data types, operators, and control structures (conditions and loops); and complete verification-oriented exercises.
• Functions and Modular Programming: Study function definitions, parameter passing, and recursive calls; cultivate modular programming thinking through comprehensive exercises; and realize code reuse and logical decomposition.
• Introduction to Classes and Objects: Gain initial exposure to object-oriented programming concepts; master class definitions, object instantiation, and member access control; and complete simple class design and application exercises.
• Arrays, Pointers, and String Processing: Proficiently use arrays to store batch data; master the principles of pointer operations and string processing methods (concatenation, search, and replacement); and solve batch data processing problems.
• Comprehensive Project Development: Independently select a Category B comprehensive task (e.g., a simple management system or a data computation tool); complete the full process of requirements analysis, algorithm design, coding implementation, debugging, and optimization; write the course-design report; and participate in the defense.
• Coding Standards and Tool Utilization: Adhere to coding standards such as identifier naming, comments, and code formatting; proficiently use programming tools for debugging; and utilize search tools to resolve compilation and linking issues.

Learning Objectives:
• Proficiently master core C++ syntax and programming techniques; independently complete procedural programming; and initially implement simple object-oriented programming.
• Develop the ability to decompose and model real-world problems; refine solutions in a top-down manner; design appropriate algorithms; and translate them into executable code.
• Proficiently operate the C++ development environment and debugging tools; rapidly locate and resolve syntactic errors and logical defects in code.
• Strictly comply with programming standards; form clear, standardized, and maintainable coding habits; and enhance code readability and reusability.
• Develop the ability to write course-design reports and conduct outcome defenses; clearly articulate project design rationale, implementation process, and core techniques.

Assessment: Regular performance (50%)Final exam (50%)

Course Materials: *C++ Programming Tutorial (3rd Edition, General Version)*, Qian Neng, ISBN: 9787302521876, Tsinghua University Press, 2019.7

4.2 Course Design of Data Processing

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Course Design of Data Processing	48	Compulsory	2.0	3.0
Semester: 2023-2024/1		**Teaching Method**: Practice(100%)		

Course Contents:
• Project Topic Selection and Feasibility Analysis: Work in groups to conceive and select a project topic; clarify project objectives (e.g., statistical data analysis, trend mining); conduct technical feasibility analysis through literature and resource retrieval; and formulate an implementation plan and technical roadmap.
• Data Collection and Storage: Learn methods for analyzing web page structures; use Python libraries such as requests and BeautifulSoup to implement web-crawling data collection; master API invocation and database operations; and complete standardized data storage (in files or databases).
• Data Preprocessing Techniques: Use the Pandas library for data cleaning (handling missing values, outliers, and duplicate data); achieve data format standardization and type conversion; optimize data quality through data integration, transformation, and reduction; and complete feature selection and dimensionality reduction.
• Data Analysis and Visualization: Self-study common data analysis algorithms; select appropriate methods for data mining according to project requirements (e.g., correlation analysis, trend analysis); use tools such as Matplotlib and Pyecharts to design visualization charts (line charts, bar charts, heatmaps, etc.); and present analysis results intuitively.
• Report Writing and Defense: Systematically organize project process documentation; write the course-design report in accordance with specifications; design a presentation PPT; complete on-site project demonstration and defense; and respond to reviewers' questions.

Learning Objectives:
• Establish data thinking; master methods for ideation, analysis, and decomposition of data-processing projects; and determine project objectives and technical solutions based on real-world needs.
• Proficiently use Python data-processing libraries; master the complete workflow and core techniques of data collection, storage, preprocessing, analysis, and visualization.
• Develop independent learning and problem-solving abilities; acquire project-required knowledge via online resources; and design reasonable solutions for issues encountered in data processing.
• Enhance written expression and oral communication abilities; clearly write project reports; and accurately present project design rationale, implementation process, and analytical conclusions.
• Cultivate teamwork awareness; efficiently complete assigned tasks within group division of labor; and collaboratively advance project implementation.

Assessment: Regular performance (50%)Final project (50%)

Course Materials: *Experimental Guidebook for Course Design of Data Processing (2022 Edition)*, Self-compiled Teaching Material, 2022

4.3 Course Project of Data Structures

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Course Project of Data Structures	32	Compulsory	2.0	3.0
Semester: 2023-2024/2		**Teaching Method**: Lecture(100%)		

Course Contents: • Comprehensive Reinforcement of Theoretical Knowledge: Consolidate core data structures such as linear lists, stacks, queues, trees, and graphs, as well as fundamental algorithms such as searching and sorting, including their principles and application scenarios, through theoretical tests and online question-bank exercises. • Implementation and Optimization of Basic Algorithms: Complete randomly assigned algorithm problems on the Touge platform, covering basic operations such as creation, insertion, deletion, and querying of data structures; submit learning notes for each problem to record implementation ideas and the optimization process. • Design and Development of Complex Systems: Independently select a complex system topic (e.g., a campus navigation system, a library management system, a Huffman encoding/decoding tool, etc.); choose appropriate storage structures and algorithms based on data-structure characteristics; and implement core functionalities. • Graphical User Interface Extension (Optional): Encourage the implementation of a graphical user interface based on core functionality to enhance user interaction experience and to train interface design and functional linkage capabilities. • Documentation Writing and Defense: Complete the course-design report according to specifications; elaborate the rationale for selecting data structures and algorithms, code implementation details, and performance analysis results; participate in on-site demonstration and defense; and present project outcomes and technical reflections.
Learning Objectives: • Develop an in-depth understanding of core theories of data structures and algorithms, and accurately identify the types of data structures and algorithms required by real-world problems. • Acquire the ability to independently implement classic data structures and algorithms using advanced programming languages, and optimize algorithm efficiency (time and space complexity) according to problem characteristics. • Master techniques for decomposition and modeling of complex problems; translate real-world requirements into data-structure and algorithm design solutions; and build functionally complete complex systems. • Enhance software documentation writing capability; write technical reports in a standardized manner; and clearly present project design, implementation process, and performance analysis. • Cultivate critical thinking and problem-solving abilities; identify potential issues during project development; and propose reasonable optimization solutions.
Assessment: Regular performance (30%)Final project (70%)
Course Materials: *Algorithms and Data Structures (C++ Language Version)*, Feng Guanghui, ISBN: 9787121356892, Electronic Industry Press

4.4 Machine Learning Curriculum Design

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Machine Learning Curriculum Design	48	Compulsory	2.0	3.0
Semester: 2024-2025/1		**Teaching Method**: Practice(100%)		

Course Contents: • Project Grouping and Topic Selection: Form groups of 2–4 members; select a project topic from categories such as regression prediction, classification prediction, clustering analysis, and deep learning (e.g., housing price prediction, sentiment analysis, image classification, etc.); and clarify the form of deliverables and core objectives. • Feasibility Analysis and Plan Formulation: Study the project background and application value; analyze data-source availability and algorithm suitability; select appropriate development platforms, software tools, and technical routes; and formulate a detailed implementation plan and division of labor. • Data Collection and Preprocessing: Obtain project data via public dataset downloads, web crawling, and other methods; use tools such as Pandas and Scikit-learn to conduct data cleaning, feature selection, and feature engineering (e.g., normalization, encoding, dimensionality reduction); and prepare high-quality data for model training. • Model Selection, Training, and Hyperparameter Tuning: Study common machine learning algorithms such as linear regression, decision trees, support vector machines, and neural networks; select suitable models according to project type; train models via train–test splitting and cross-validation; and optimize hyperparameters using grid search and random search methods. • Model Deployment and System Development: Integrate the trained model into a web or desktop application to realize practical prediction functionality; design a simple user interaction interface; and complete system function testing and performance optimization. • Report Writing and Defense: Organize full-process project documentation (data description, model design, code implementation, test results); write a standardized course-design report; prepare a PPT and participate in the on-site defense; and submit source code, datasets, and demonstration videos and other materials.
Learning Objectives: • Master the complete development process of machine learning projects, and complete topic selection, solution design, and implementation based on real-world needs. • Proficiently use machine learning tools and libraries, and develop core skills in data preprocessing, feature engineering, model training, and hyperparameter optimization. • Understand the applicable scenarios, advantages, and limitations of different machine learning algorithms, and select the optimal model solution according to data characteristics and project requirements. • Develop the ability to transform machine learning models into practical application systems, and understand the basic processes and key technical points of model deployment. • Enhance teamwork, technical documentation writing, and oral presentation abilities; clearly present project outcomes and technical details; and respond to professional questions.
Assessment: Regular performance (60%)Final project (40%)
Course Materials: *Experimental Guidebook for Machine Learning Curriculum Design (2023 Edition)*, Self-compiled Teaching Material, 2023

4.5 Intelligent System Curriculum Design

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Intelligent System Curriculum Design	48	Compulsory	2.0	3.0
Semester: 2024-2025/2		**Teaching Method**: Practice(100%)		

Course Contents: • Competition Project Selection and Team Formation: Form groups of 2–4 members; select an appropriate intelligent-system competition project from platforms such as Aliyun Tianchi and Kaggle; complete competition registration; and clarify task requirements and scoring criteria. • Project Requirements Analysis and Solution Design: Thoroughly interpret the competition topic; analyze dataset characteristics and core tasks (e.g., prediction, classification, optimization); identify innovation points and technical challenges; select data processing methods, core algorithms, and model architectures; and formulate a detailed implementation plan and division of labor. • Data Processing and Feature Engineering: Download the competition dataset; use Python-related libraries to complete data cleaning, missing-value imputation, and outlier handling; design feature extraction and optimization schemes for the competition task (e.g., feature combinations, dimensionality reduction); and improve data quality. • Model Design, Training, and Optimization: Select appropriate machine learning/deep learning models (e.g., complex neural networks, ensemble learning models) according to the competition task; complete model construction, training, and iterative optimization; validate model performance via the competition platform; and continuously adjust parameters and architectures to improve scores. • System Integration and Testing/Deployment: Integrate the optimized model into the system framework required by the competition; complete functional testing, performance testing, and compatibility testing; submit system outcomes according to competition requirements; and record platform validation scores. • Report Writing and Defense: Summarize the competition participation process; write the course-design report (including requirements analysis, solution design, implementation process, optimization strategies, and competition results); prepare a PPT and participate in the on-site defense; and submit source code, datasets, and demonstration videos and other materials.
Learning Objectives: • Develop the ability to extract core requirements and technical challenges from competition projects, and formulate scientifically grounded implementation plans and technical routes. • Proficiently master core techniques in intelligent system development, including data processing, model design, and training optimization, and continuously improve system performance in competition scenarios. • Understand basic methods for intelligent system deployment and performance monitoring, and optimize system functionality and stability based on testing results and competition feedback. • Cultivate teamwork and problem-solving abilities; efficiently divide labor and collaborate under competitive pressure; and overcome technical bottlenecks encountered in development. • Enhance technical report writing and outcome presentation abilities; clearly explain the design rationale, implementation process, innovation points, and participation gains of the competition project.
Assessment: Competition score (100%)
Course Materials: *Provides a list of recommended competition platforms (for project topic selection and practice).*

4.6 Comprehensive Course Design of Specialty

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Comprehensive Course Design of Specialty	48	Compulsory	2.0	3.0
Semester: 2025-2026/1		**Teaching Method**: Practice(100%)		

Course Contents: • Project Grouping and Topic Justification: Form groups of 2–4 members; select a comprehensive project topic from areas such as intelligent systems, computer vision, and natural language processing (e.g., intelligent recognition systems, data analysis platforms, robotics applications, etc.); justify the project's social value and technical feasibility through literature review and market research; and determine project objectives and deliverables. • Overall System Design: Complete requirements analysis and architecture design; clarify system functional modules (e.g., data acquisition module, processing module, model module, interaction module); select development platforms, core technologies, and tool stacks; and formulate a detailed development plan and division of labor. • Core Module Implementation: • Data Layer: Complete the acquisition, storage, and preprocessing of multi-source data (structured and unstructured) to ensure data support for core system functionality. • Algorithm Layer: Integrate machine learning, deep learning, or data-structure-related algorithms to implement core business logic (e.g., recognition, prediction, decision-making). • Application Layer: Design a user interaction interface (web/mobile/desktop); realize linkage between algorithms and the interface; and ensure system usability. • System Testing and Optimization: Formulate a comprehensive testing plan (functional testing, performance testing, compatibility testing); identify system vulnerabilities and performance bottlenecks; and enhance system stability and operational efficiency through algorithm optimization, code refactoring, and parameter tuning. • Outcome Summary and Defense: Write a complete course-design report, detailing project background, requirements analysis, system design, implementation process, test results, and application prospects; prepare a high-quality PPT; complete on-site presentation and defense; and submit a full set of materials including source code, datasets, and demonstration videos.
Learning Objectives: • Be able to comprehensively apply the core knowledge of the Intelligent Science and Technology major (programming, data processing, machine learning, system development, etc.) to solve complex engineering problems. • Develop end-to-end capabilities for complex intelligent systems, including requirements analysis, architecture design, module development, and integration testing; and independently formulate reasonable technical solutions and development plans. • Proficiently use professional development tools and technology stacks; flexibly address technical difficulties during project development; and develop independent learning and technological innovation abilities. • Cultivate teamwork and project management awareness; communicate effectively within the group, divide labor appropriately, and collaboratively advance the project to ensure timely and high-quality completion. • Possess strong abilities in technical documentation writing, outcome presentation, and academic defense; and clearly and professionally present project outcomes and technical reflections.
Assessment: Regular performance (40%)Final project (60%)
Course Materials: *Provides a list of specialty comprehensive course design topics (Self-compiled).*

5 M500 Programming Languages and Software Development

Course Information Overview

Course Name	Credits	ECTS
UI Design	2.5	3.75

Python Programming Design	3.0	4.5
Object-Oriented Programming in Java	3.0	4.5
Matlab language and its application	2.5	3.75
Java Web Development Technology	3.0	4.5
Mobile Application Development Technology	2.5	3.75
Total	**16.5**	**24.75**

5.1 UI Design

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
UI Design	48	Elective	2.5	3.75
Semester: 2022-2023/1		**Teaching Method**: Lecture (66.7%) and Lab (33.3%)		

Course Contents:
• HTML5 Core Technologies: Usage of semantic structural elements (header, nav, section, footer); HTML5 form API (new input types and attributes); Canvas API for basic graphics rendering (paths, rectangles, text, images); Web Storage (localStorage and sessionStorage); Drag-and-Drop API; Geolocation API; basic usage of audio and video elements.
• CSS3 Core Skills: Common selectors (tag, class, id, pseudo-class); box model and layout mechanisms (Flexbox and Grid); background, text, and typography styling; 2D/3D transformations and keyframe animations; responsive layout design using media queries for mobile and PC adaptation.
• JavaScript and DOM Interaction: JavaScript fundamentals (variables, data types, operators); control statements and functions; DOM element selection, creation, and modification; event handling (mouse, keyboard, and form events); implementation of basic interactive behaviors such as image carousels and form validation.
• Prototyping and Practical Tools: Basic operations of Figma for interface prototyping; integration of visual design drafts with front-end implementation; export and slicing of design assets (PNG/JPG) for web development.
• Basic UI Design Principles for Web Interfaces: User-centered interface logic; fundamental color matching and typography principles; basic micro-interaction design (e.g., button click feedback and simple page transitions).

Learning Objectives:
• Master core front-end technologies (HTML5, CSS3, and JavaScript) and be able to implement functional web interface modules using common HTML5 APIs such as forms, Canvas, and Web Storage.
• Apply CSS3 layout, transformation, and animation techniques to develop responsive web pages compatible with both mobile and desktop environments.
• Use JavaScript and DOM manipulation techniques to realize interactive web behaviors, including event handling, form validation, and simple dynamic content updates.
• Use Figma for basic UI prototyping and asset export, and understand the workflow from interface design to front-end implementation.
• Complete small-scale practical projects (e.g., club website layout, user registration page, or interactive image carousel) by integrating HTML5, CSS3, and JavaScript technologies.

Assessment: Regular performance (15%); Lab performance (35%); Final exam (50%)

Course Materials: *HTML5 Web Front-End Design (2nd Edition – Microlecture Video Edition)*, Zhou Wenjie (Editor-in-Chief), ISBN: 9787302550635, Tsinghua University Press

5.2 Python Programming Design

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Python Programming Design	64	Compulsory	3.0	4.5
Semester: 2022-2023/2		**Teaching Method**: Lecture (50%) and Lab (50%)		

Course Contents:
• Fundamental Syntax and Environment: Fundamental concepts of the Python language; configuration of Anaconda/PyCharm development environments; module/package management; the name attribute and the help/dir utilities.
• Language Core: Data types, operators, and control statements (if/while/for).
• Data Structures: Basic operations on lists, tuples, dictionaries, and sets; comprehensions (list/dict).
• Strings and Regular Expressions: String formatting and built-in methods; basic regular-expression syntax and simple matching.
• Functions and Advanced Topics: Function definition/calling and variable scope; Lambda functions and simple recursive functions.
• Classes and Objects: Basic class definitions, attributes, and methods; simple inheritance and exception handling.
• Files and Office Data: File read/write operations; Excel document processing (openpyxl basics); directory/path management.
• Web Data Acquisition: Basic HTML parsing; fundamental data collection with urllib/BeautifulSoup4; compliance norms for lawful data acquisition.
• Data Analysis and Visualization: Basic computations with Numpy; fundamental plotting with Matplotlib and simple interaction (chart zooming, legend customization); Pandas data reading and simple preprocessing.
• Version Control and Data Processing: Core Git concepts (commit/pull/branch); basic operations with the GitPython library (repository cloning, code commits); JSON data processing (core functions of the json module: dumps/loads/dump/load).
• Environment and Practice: Virtual-environment configuration (using the venv module to isolate project dependencies); practical projects (EBANK bank teller ATM system, 12 verification experiments, simple API-calling practice).
• Foundations of Advanced Applications: Basic computer vision (OpenCV image filtering/simple segmentation); understanding the installation of deep learning frameworks (Tensorflow/Keras) and invocation of pretrained models.

Learning Objectives:
• Master fundamental Python syntax, commonly used libraries (Numpy/Pandas/Matplotlib/OpenCV basics/GitPython), and development environments; become proficient in configuring and using virtual environments.
• Be able to use Python to solve simple engineering problems (e.g., data processing, small-scale project development, API invocation, JSON format transformation).
• Develop practical capabilities in Excel processing, basic web data collection, regular expressions, and Git-based version control.
• Be able to independently complete small projects such as the EBANK teller ATM system; strengthen programming skills through experiments; customize Matplotlib chart styles and implement basic interactions.
• Master the creation and use of Python virtual environments to isolate project dependencies; be able to use the json module to complete bidirectional conversion between Python data and JSON formats.

Assessment: Regular performance (30%); Lab performance (20%); Final exam (50%)

Course Materials: *Python Programming Tutorial (3rd Edition – Microlecture Video Edition)*, Yang Nianhua, Liu Qing, Zheng Jiming (Editors-in-Chief), ISBN: 9787302626884, Tsinghua University Press

5.3 Object-Oriented Programming in Java

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Object-Oriented Programming in Java	64	Elective	3.0	4.5
Semester: 2023-2024/2		**Teaching Method**: Lecture (50%) and Lab (50%)		

Course Contents:
• Fundamental Syntax and Environment: Java language characteristics; configuration of development environments (Eclipse/IntelliJ); primitive data types and operators.
• Control Structures: Basic usage of structured control statements (if/switch/for/while).
• Object-Oriented Core: Classes and objects; access modifiers; fundamental concepts of inheritance, polymorphism, and encapsulation; introduction to abstract classes and interfaces.
• Data Processing: One-dimensional arrays and string operations; basic use of the collections framework (ArrayList/HashMap); advanced use within the collections framework (basic use of LinkedList and HashSet).
• Exceptions and I/O: Basic exception-handling structures; simple operations for file reading and writing.
• GUI Programming: Basic Swing components (text fields/buttons); use of simple layout managers.
• Testing and Debugging: Fundamentals of JUnit5 unit testing (@Test annotation, simple assertions); Java coding conventions (naming/commenting rules); basic debugging techniques (breakpoint debugging in Eclipse/IntelliJ).
• Advanced Data Processing: Introduction to the Stream API (basic operations such as filtering, sorting, and traversal).
• Practice: 16 verification experiments on the Touge teaching platform (including OOP, GUI, file I/O, and unit testing).

Learning Objectives:
• Master core Java object-oriented concepts (classes/inheritance/encapsulation), the collections framework (including LinkedList/HashSet), and fundamentals of JUnit5 unit testing.
• Be able to use Swing to develop simple GUI interfaces; master file I/O and basic exception handling; and use the Stream API to simplify basic processing of collection data.
• Be able to follow basic Java coding conventions; strengthen programming skills through Touge experiments; master basic breakpoint-debugging methods; and efficiently locate and resolve code errors.
• Understand the applicable scenarios of different collections and address simple coding problems and data-processing needs.

Assessment: Regular performance (50%); Final exam (50%)

Course Materials: *Java Programming (4th Edition)*, Xin Yunwei, Rao Yimei (Editors-in-Chief), ISBN: 9787302472684, Tsinghua University Press

5.4 Matlab language and its application

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Matlab language and its application	48	Elective	2.5	3.75
Semester: 2023-2024/2		**Teaching Method**: Lecture (66.7%) and Lab (33.3%)		

Course Contents: • MATLAB Fundamental Operations: Use of the interface and script-file writing; basics of code commenting and debugging. • Core Applications: Numerical computation (systems of linear equations, interpolation, numerical integration); fundamentals of signal processing (simple filtering); data visualization (basic chart plotting). • Advanced Data and Plotting: Extended data import/export (Excel/CSV import); customization of plotting styles (line color, line style, marker settings; axis labels and legend insertion). • Automation and Layout: Script automation (loop-based processing of multiple groups of similar data and batch generation of computational results); basics of multi-figure layout (use of the subplot function to display multiple charts in one window). • Practice: Exercises from the textbook after-class problems; hands-on practice of simple computational tasks and mathematical modeling cases (e.g., statistical analysis of stock data).
Learning Objectives: • Master the basic use of MATLAB in numerical computation and signal processing; become familiar with Excel/CSV data import/export and multi-figure layout methods. • Be able to use MATLAB to complete simple computational tasks such as solving differential equations, basic filtering, and data visualization; customize plot styles to improve readability. • Be able to write automated scripts to batch-process multiple groups of data and output results; complete after-class exercises and simple mathematical modeling cases. • Be able to produce process outputs such as lab reports and efficiently present computational and analytical results.
Assessment: Regular performance (45%); Final exam (55%)
Course Materials: *MATLAB Basic Tutorial (5th Edition)*, Xue Shan (Editor-in-Chief), ISBN: 9787302598065, Tsinghua University Press

5.5 Java Web Development Technology

Course Title	**Hours**	**Course Type**	**Credit**	**ECTS**
Java Web Development Technology	64	Elective	3.0	4.5
Semester: 2024-2025/1		**Teaching Method**: Lecture (50%) and Lab (50%)		
Course Contents: • Web Development Fundamentals: Basics of HTML/CSS/JavaScript (front-end page layout and simple interactions); introduction to the HTTP protocol; configuration and deployment of the Tomcat server. • Core Technologies: JSP/Servlet fundamentals; JDBC database access; introduction to the MVC pattern. • Frameworks and Data: Fundamentals of the SSM framework (Spring/SpringMVC/MyBatis); basic SpringBoot configuration (application.yml configuration); basic SpringBoot integration with MyBatis-Plus (rapid generation of CRUD interfaces); simple RESTful API development. • Databases and Data Formats: Basic optimization in MySQL (introduction to indexing); processing of XML/JSON data formats. • Functional Extensions: Simple file upload functionality (adding an image upload module to a blog system); HTTP status codes and basic error handling (simple adaptation of 404/500 pages). • Practical Project: Team-based development of a simple blog system (including user management, article publishing, front-end page interaction, and image upload functionality).				

Learning Objectives: • Master core Java Web technologies (JSP/Servlet, JDBC, SSM/SpringBoot fundamentals), front-end basics, Tomcat deployment methods, and integration techniques for MyBatis-Plus. • Be able to implement basic database interactions (CRUD) and optimization; process XML/JSON data formats; and add simple file upload functionality to Web projects. • Understand the meanings of common HTTP status codes; implement basic error-page adaptation; and integrate various functions into a Web system. • Be able to participate in the development and delivery of small team projects (e.g., a blog system); understand basic front-end/back-end collaboration workflows; and improve project practicality.
Assessment: Regular performance (20%); Lab performance (30%); Final exam (50%)
Course Materials: *JSP & Servlet Study Notes (3rd Edition) — From Servlet to Spring Boot*, Lin Xinliang, ISBN: 9787302522454, Tsinghua University Press

5.6 Mobile Application Development Technology

Course Title	Hours	Course Type	Credit	ECTS
Mobile Application Development Technology	48	Elective	2.5	3.75
Semester: 2024-2025/1		**Teaching Method**: Lecture (66.7%) and Lab (33.3%)		
Course Contents: • Development Fundamentals: Android Studio environment configuration; Android project structure and UI layout (ConstraintLayout/LinearLayout); fundamentals of APP permission management (storage/network permissions). • Core Functions: Use of basic UI components; simple data storage (SharedPreferences/SQLite); understanding of the Activity lifecycle. • Components and Functions: Fundamentals of the Intent component (page navigation and simple data passing); basic use of third-party libraries (e.g., Glide for image loading); simple notification functionality (scheduled reminders for a memo app and notification-bar display). • Adaptation and Testing: APP UI adaptation (basic adaptation for screen sizes); basic APP testing (real-device debugging, Logcat log viewing, and troubleshooting). • Practical Project: Independently develop a memo APP (including note editing, local storage, image attachments, and scheduled notification functionality); deploy the APP to a mobile device and conduct basic testing.				
Learning Objectives: • Master basic operations in Android Studio, UI layout, APP permission management, and the fundamental use of the Intent component. • Be able to implement simple data storage, third-party library integration, page navigation and data passing, and add scheduled notification functionality in mobile applications. • Be able to independently complete the development, adaptation, and deployment of small mobile applications such as a memo APP; master real-device debugging and log-based troubleshooting methods. • Understand the core workflow of mobile application development; be able to address basic development issues; and improve APP functional completeness and user experience.				
Assessment: Regular performance (40%); Final project (60%)				
Course Materials: *Android Studio Mobile Development Tutorial (1st Edition)*, Xiao Kun, Wu Zhixiang et al. (Compilers), ISBN: 9787121356892, Electronic Industry Press				